

2203. Minimum Weighted Subgraph With Required Paths (Google)

we need to return minimum weight of a subgraph of the graph such that it is possible to reach dest from both src1 and src2 via a set of edges, in case more than -1.

Observations & Logic

Obs 1

Suppose both paths meet at some node X then answer becomes

$$\text{src1} \rightarrow X + \text{src2} \rightarrow X + X \rightarrow \text{dest}$$

So we need `dist1[i]` , `dist2[i]` , `dist3[i]` and take minimum over all i.

now for i to dest either we can do

```
for(int i=0 ; i<n ; i++) {  
    djstras(i);  
    take distance to dest  
}
```

this will be $O(n * E \log V)$

Obs 2

let's run for every i then

```
1 → 4
2 → 4
3 → 4 → Fixed
4 → 4
```

hence we make (everyone → dest) to (dest → everyone).

Dijkstra's power is given one source find distance to all nodes.

ASK

what graph can I build such that starting from dest no equivalent to ending at dest, is path cost remain same.

C++ Implementation

```
class Solution {
public:

    vector<long long> djik(vector<vector<pair<int,long long>>>& adj,int src){

        int n=adj.size();
        priority_queue<pair<long long,int>,vector<pair<long long,int>>,greater<pair<long long,int>>>
        vector<long long> dist(n,LLONG_MAX);
        dist[src]=0;
        pq.push({0,src});

        while(!pq.empty()){
            int currnode=pq.top().second;
            long long currdis=pq.top().first;
            pq.pop();

            if(currdis>dist[currnode]) continue;

            for(auto &it:adj[currnode]){
                int newnode=it.first;
                long long cost=it.second;
```

```

        long long newdis=currdis+cost;
        if(newdis<dist[newnode]){
            dist[newnode]=newdis;
            pq.push({newdis,newnode});
        }
    }

    return dist;
}

long long minimumWeight(int n, vector<vector<int>>& edges, int src1, int src2, int dest) {

    vector<vector<pair<int,long long>>> adj(n);
    vector<vector<pair<int,long long>>> revadj(n);

    for(auto it:edges){
        int u=it[0];
        int v=it[1];
        long long w=it[2];

        adj[u].push_back({v,w});
        revadj[v].push_back({u,w});
    }

    vector<long long> dist1=djk(adj,src1);
    vector<long long> dist2=djk(adj,src2); //min cost to go on any node from src1 and src2
    vector<long long> dist3=djk(revadj,dest);
    long long ans=LLONG_MAX;
    for(int i=0;i<n;i++){
        //lets try all nodes as intersection points

        if(dist1[i]==LLONG_MAX || dist2[i]==LLONG_MAX || dist3[i]==LLONG_MAX){ //intersection point not possible
            // sabhi points ko try krna hoga
            continue;
        }

        ans=min(ans,dist1[i]+dist2[i]+dist3[i]);
    }

    if(ans==LLONG_MAX) return -1;
    else return ans;
}

};

```